

01_variables

August 6, 2026

1 01 - PYTHON VARIABLES

Name: Vamshi / Date: 05/08/2026

- Variable is a name assigned to a value. This name refers to a memory location.
- Python variables are also called as objects.
- Syntax: **variable = value**

variable —> name assigned for memory location & value —> datatype

Rules to define a python variable name:

1. Variable names must ****start with a letter or an underscore (_)****.
2. They **cannot start with a digit/number** but can end with digit/number.
3. It can contain ****letters, digits, and underscores (_)**** but, only if above 2 rules are met.
4. **Cannot contain spaces or special characters.**
5. They are **case-sensitive** (python, Python, and PYTHON are different variables).
6. It **cannot be Python keywords**. =====> Keywords are also known as reserved words.

```
[5]: n=10 # Integer value
n
```

```
[5]: 10
```

```
[6]: 10=n # Not Valid - syntax error - Rule 2
```

```
Cell In[6], line 1
    10=n # Not Valid - syntax error
    ^
SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='
```

```
[8]: name = 'stark' # String value
name
```

```
[8]: 'stark'
```

```
[9]: NAME # Name Error - Rule 5, becoz variable NAME is not present (its case_
    ↪ensitive - name and NAME are not the same)
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[9], line 1  
----> 1 NAME #  
  
NameError: name 'NAME' is not defined
```

```
[10]: 1a = 25 # Syntax Error - Rule 2
```

```
Cell In[10], line 1  
    1a = 25  
    ^  
SyntaxError: invalid decimal literal
```

```
[11]: name@ = 'tony' # Syntax Error - Rule 4  
name@
```

```
Cell In[11], line 1  
    name@ = 'tony' # Name Error - Rule 4  
    ^  
SyntaxError: invalid syntax
```

```
[12]: name_1 = 'tony' # Rule 3  
name_1
```

```
[12]: 'tony'
```

```
[13]: for = 45 # Syntax Error - Rule 6 (kw cannot be used as variable name)
```

```
Cell In[13], line 1  
    for = 45 # Rule 6  
    ^  
SyntaxError: invalid syntax
```

```
[15]: FOR = 45  
FOR
```

```
[15]: 45
```

```
[23]: val = 53.45 # Float value
      val
```

```
[23]: 53.45
```

```
[33]: val_1 = True # Boolean value
      val_1
```

```
[33]: True
```

To display the output on the screen, print() function is used.

```
[19]: print(FOR)
      print(name_1)
      print(name)
      print(n)
```

```
45
tony
stark
10
```

****The type() function is used to identify the datatype of the variable****

```
[35]: print(type(n))
      print(type(name))
      print(type(val))
      print(type(val_1))
```

```
<class 'int'>
<class 'str'>
<class 'float'>
<class 'bool'>
```

To check the list of all keywords and print total number of keywords:

```
[37]: import keyword
      keyword.kwlist # Lists all the key words
```

```
[37]: ['False',
      'None',
      'True',
      'and',
      'as',
      'assert',
      'async',
      'await',
      'break',
      'class',
      'continue',
      'def',
```

```
'del',  
'elif',  
'else',  
'except',  
'finally',  
'for',  
'from',  
'global',  
'if',  
'import',  
'in',  
'is',  
'lambda',  
'nonlocal',  
'not',  
'or',  
'pass',  
'raise',  
'return',  
'try',  
'while',  
'with',  
'yield']
```

```
[38]: kw = len(keyword.kwlist)  
print("The total number of keywords are", kw)
```

The total number of keywords are 35

```
[ ]:
```